

Packet Network Performance

- * Three main metrics

- ** Throughput – rate at which we can move data across a network.
- ** Delay – Time required for packet to go from end-to-end.
- ** Data loss – Fraction of data that is corrupted or discarded by network.

- * What constrains network performance?

- ** Physical infrastructure – data rate of network links, propagation delay of signals, interference/noise.
- ** Competing traffic – creates queuing delays, packet loss due to queue overflow.

- * Providing performance guarantees

- ** Requires reserving network capacity for specific traffic streams.
- ** Can be done in packet networks and is done with many private networks- no support for this in public internet.

Delays in Networks

- * Speed-of-light delay

- ** In fiber, about 130000 miles/s.

- ** NY to LA is about 5000 km, so 23ms. (but network links add distance making 30 ms a better estimate).

- * Transmission delay – determined by data rate of links

- ** On 1 Mb/s link, takes 10 ms to send a 10000 bit packet.

- ** On 10 Gb/s link takes 1 micro sec, and on 50 Kb/s links takes 200 ms.

- * Queuing delays

- ** Each link in a network is preceded by a queue.

- *** Implemented using memory.

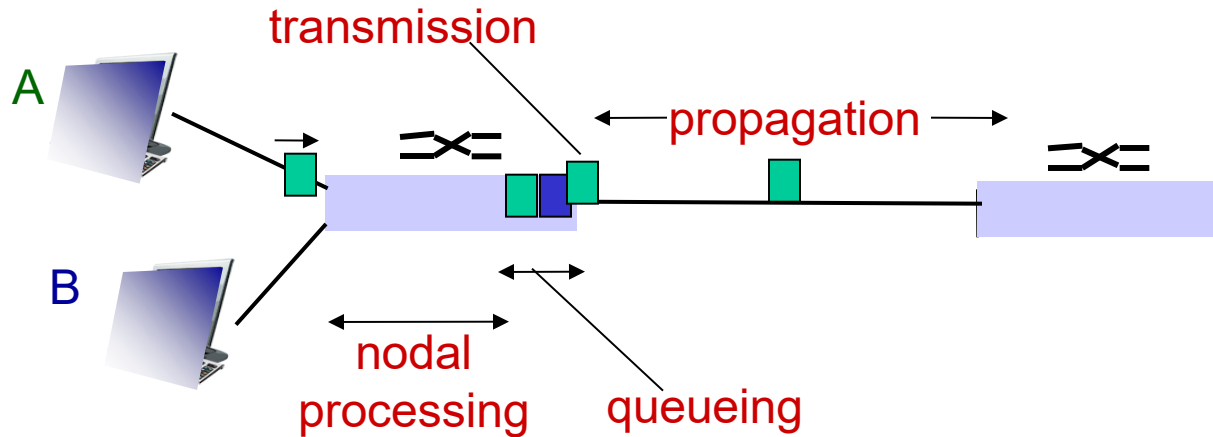
- *** Packets must wait for those ahead of them in the queue.

- ** Depends on number/length of packets in queue and link speed.

- ** For lightly loaded links, rarely more than 10 packets in queue.

- ** On overloaded links, queuing delays may exceed 100 ms.

Delays in Networks



$$\text{Total Delay} = P_D + Q_D + T_D + \text{Prop}_D$$

$T_D = L/R$, Where L = Packet length, R = Link bandwidth (bps)

$\text{Prop}_D = d/s$ Where d = distance of link, s = propagation speed in medium

$Q_D = a L/R$ Where L = Packet length, R = Link bandwidth (bps) and a = Average packet arrival rate.

- * $La/R \sim 0$: avg. Queuing delay is small.

- * $La/R \rightarrow 1$: avg. Queuing delay is large.

- * $La/R > 1$: more work is arriving than can be serviced, avg. Delay is infinite.

Other Sources of delay

- * **Packetization delay** – time to acquire data to be sent

- ** voice typically encoded at 32 Kb/s (4 bytes per milliseconds).

- ** So, if we limit packetization delay to 20 ms, each packet carries 80 bytes of voice data.

- ** An internet packet has about 50 bytes of “overhead” (40% of total).

- ** Can reduce overhead percentage by increasing delay.

- ** For telephony, need to limit delay for good user experience.

- * **Component processing delay**

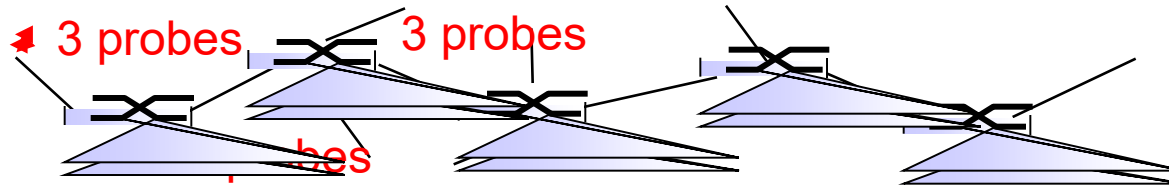
- ** Components like routers, wireless access points and hosts consume some time processing packets.

- *** for routers this includes checking packets for errors and determining where they should go next.

- ** added to other delays (like queuing , transmission).

- ** typically fairly small: 1-10 micro seconds for routers, 10-50 micro seconds for hosts.

End-To-End Delay



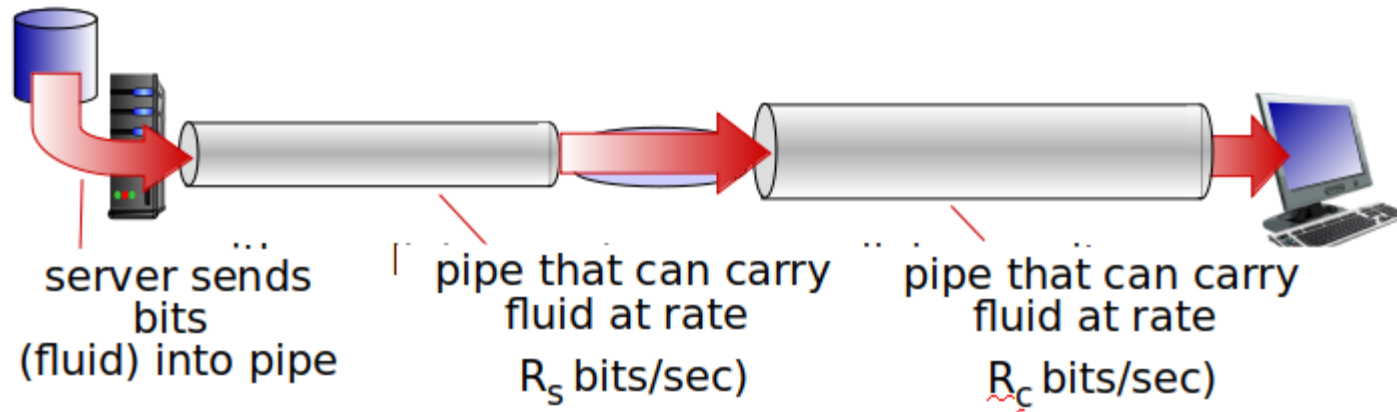
```
Activities Terminal Sat 08:45 raj@raj-HP-ProBook-440-G5: ~
File Edit View Search Terminal Help
raj@raj-HP-ProBook-440-G5:~$ traceroute www.bits-pilani.ac.in to www.eurecom.fr
traceroute to www.eurecom.fr (193.55.113.240), 64 hops max
 1  10.40.0.1  1.863ms  1.618ms  1.862ms
 2  10.1.0.10  1.700ms  3.852ms  1.585ms
 3  103.210.49.129  2.116ms  1.764ms  1.946ms
 4  103.123.50.37  3.877ms  4.071ms  1.968ms
 5  45.116.0.133  4.715ms  4.238ms  2.035ms
 6  182.75.249.57  12.204ms  11.833ms  11.014ms
 7  182.79.222.33  332.116ms  307.234ms  307.029ms
 8  173.205.61.25  310.972ms  303.332ms  306.844ms
 9  213.200.119.214  307.220ms  336.363ms  278.072ms
10  77.67.123.210  306.910ms  307.232ms  306.998ms
11  193.51.180.55  307.052ms  307.443ms  306.451ms
12  193.51.177.169  307.269ms  306.828ms  308.036ms
13  193.51.177.184  307.512ms  306.257ms  306.728ms
14  193.51.177.21  312.651ms  305.101ms  304.835ms
15  193.51.187.17  305.631ms  320.878ms  294.832ms
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
31  * * *
32  * * *
33  * * *
34  * * *
35  * * *
36  * * *
37  * * *
```

Packet loss

- * Bits corrupted on links
 - ** very rare in fiber – typically, less than 1 bit in 10^{12} have errors.
 - ** Far more frequent in wireless – one in 10^3 to one in 10^5 .
 - *** also highly variable: interference, distance from access point.
 - *** can reduce using variety of coding techniques.
 - ** Electrical signals over wire – intermediate error rates.
 - ** Errors detected by routers, which discard corrupted packets.
- * Losses due to lack of space in packet queues
 - ** can be kept close to zero if traffic flows are regulated to avoid overloads.
 - ** but very common in internet (packet loss rates of 2-20% are not unusual).
- * Components like routers may corrupt packet data
 - ** Extremely rare in correctly functioning components.

Throughput

- * Rate (bits/time unit) at which bits transferred between sender/receiver.
- * Instantaneous : rate at given point in time
- * Average : rate over longer period of time



Suppose Host A wants to send a large file to Host B. The path from Host A to Host B has four links, of rates $R_1 = 500$ kbps, $R_2 = 2$ Mbps, $R_3 = 1$ Mbps and $R_4 = 3$ Mbps. Assuming no other traffic in the network, what is the throughput for the file transfer?

Answer: 500 kbps

Consider a LAN with a maximum distance of 2km. At what bandwidth would propagation delay (at a speed of 1×10^8 m/s) equals to transmit delay for 100-byte packets? What about 512-byte packets?.

Solution

prop delay = distance / prop speed

$$= 2 * 10^3 \text{ m} / 1 * 10^8 \text{ mps} = 1 * 10^{-5} \text{ s} = 20 \text{ us}$$

trans time = size / bandwidth

For an 800 bit packet

$$= 800 \text{ b} / x \text{ Mbps} = 20 \text{ us}$$

$$x = 40 \text{ Mbps} = 5 \text{ MBps}$$

For a 4096 bit packet

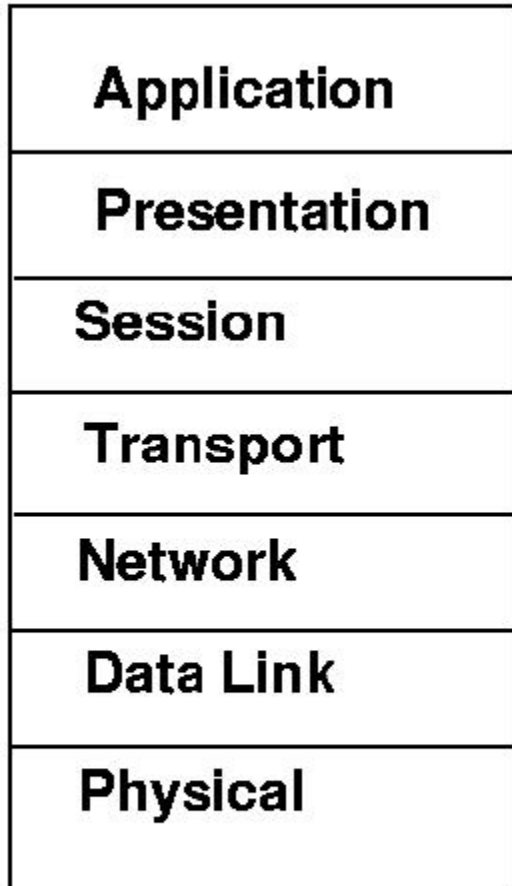
$$x = \text{approx } 204.5 \text{ Mbps} = 25.6 \text{ MBps}$$

OSI Reference Model

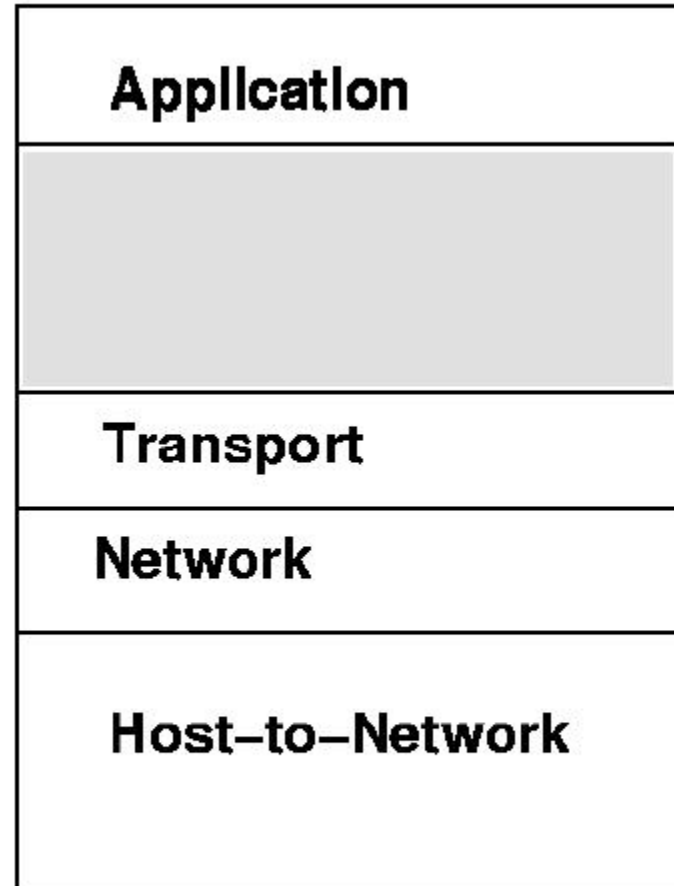
The International Standards Organization (ISO) proposal for the standardization of various protocols used in computer networks is called the Open Systems Interconnection Reference Model.

Although the OSI model is just a model (not a specification), it is generally regarded as the most complete model.

OSI Reference Model



The OSI model



The TCP/IP model

Application Layer:

- The Application Layer is where users interact with the system.
- Applications such as Telnet, FTP, SMTP, mail, http, live at this layer.
- The Application layer is responsible for coding the message.
- An Application Layer Header is added to the data.

Presentation Layer:

- Translation, cryptography, and code conversion take place at this level.
- A common Language is determined.
- Data Compression occurs.
- Data Encryption occurs.
- Big endians, little endians: data can be encoded prior to transmission data can be decoded upon receipt.

Session Layer:

- Managing dialog control: Traffic can go in both directions at the same time, or in only one direction at a time.
- Token management: both sides do not attempt the same operation at the same time.
- Synchronization: Insertion of checkpoints to determine portions of transmissions.

Transport Layer:

- Frees upper layers from the concerns of achieving data transfer.
- The transport layer deals with accepting data from its higher layer, splitting it up into smaller units if necessary, and then passing these units to the network layer.
- This layer also ensures that all of the units arrive correctly at the other end, and that all of this is done efficiently and in such a way as to isolate the upper layers.
- Design issues include:
 - **Connection orientation**
 - **Flow control:** The issue of ensuring that a fast machine does not swamp a slow machine, must be considered.
 - **The possible use of multiple connections:** Normally, the transport layer creates a distinct network connection for each connection required by the session layer.
 - **The possible use of multiplexing:** If high throughput is required, and multiple connections are too expensive, then may multiplex several connections onto the same connection.
 - Controlling congestion.
 - **What type of service to provide:** point-to-point isolated messages
broadcast

Network Layer:

- The network layer deals with controlling the operation of the subnet.
- The subnet contains switching elements and hosts between two computers.
- Design issues include:
 - How to route packets from a source to a destination (routing). There may be many paths depending upon network topology.
 - Controlling congestion
 - Accounting functions
 - Different addressing and size requirements, and protocols used when interconnecting heterogeneous networks (translation between different network types).
 - packet headers
 - virtual circuits

Data Link Layer:

- The data link layer deals with taking a raw transmission facility and transforming it into a line that appears free of undetected transmission errors to the network layer.
- Design issues include:
 - **Frame boundaries:** The data link layer performs its primary task by breaking the input data into frames.
 - **Possible errors in frame data:** A noise burst on the line can completely destroy a frame.
 - **Transmission rate:** Some traffic regulation mechanism must be employed to ensure that a very fast machine doesn't send data at such a high rate that it "swamps" the receiving machine.
 - How to resolve competition for the line when full duplex transmission is used.

Physical Layer:

- The physical layer deals with the transmission of raw bits over a communication channel.
- Mechanical, electrical and procedural interfaces, the transmission medium.

Some apps

- E-mail
- Web
- text messaging
- remote login
- P2P file sharing
- multi-user network games
- streaming stored video
(YouTube, Hulu, Netflix)
- voice over IP (e.g., Skype)
- real-time video conferencing
- social networking
- search

Creating a network app



write programs that:

- run on (different) *end systems*
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices:



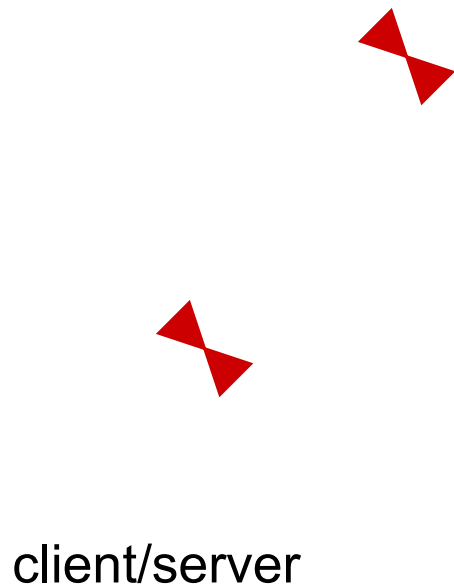
- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation

Application architectures

possible structure of applications:

- Client-server
- peer-to-peer (P2P)

Client-server architecture



server:

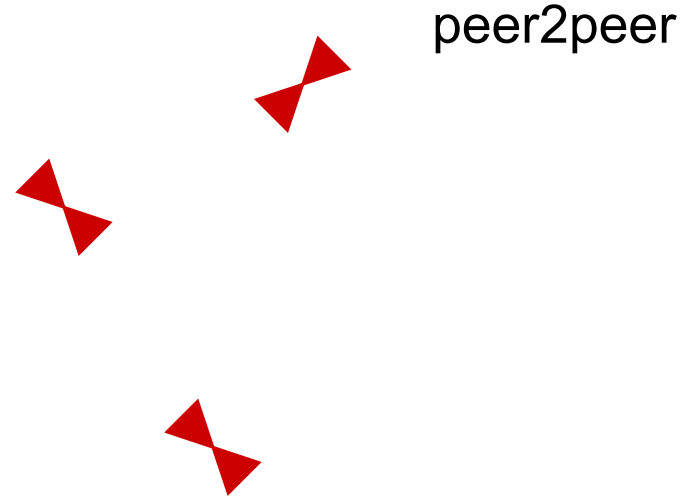
- always-on host
- permanent IP address
- data centers for scaling

clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

P2P architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management



Processes communicating

clients, Servers

process: program running within a host

- within same host, two processes communicate using *inter-process communication* (defined by OS)
- processes in different hosts communicate by exchanging *messages*

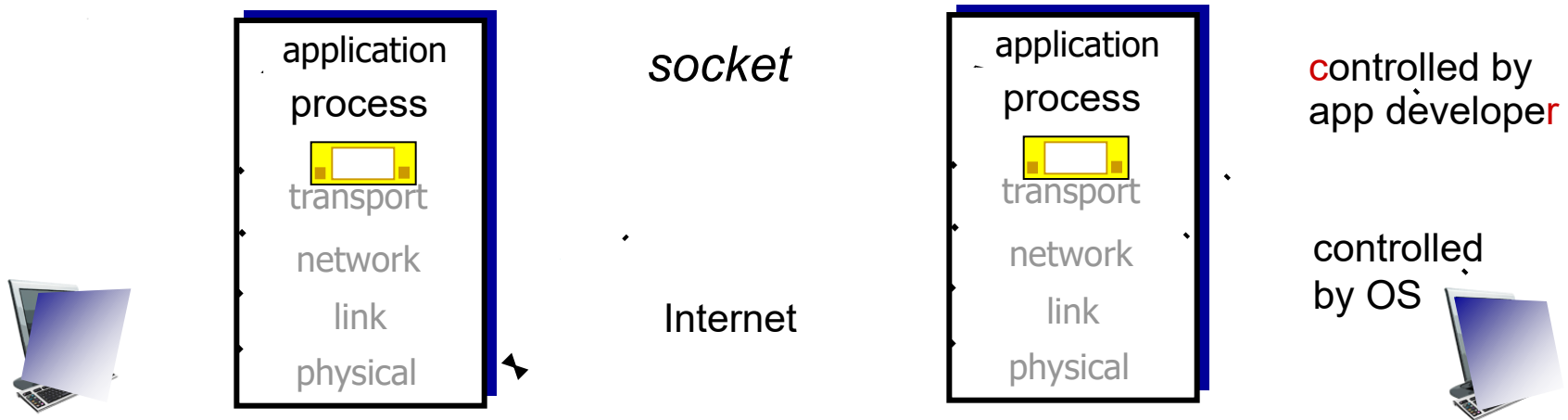
client process: process that initiates communication

server process: process that waits to be contacted

- aside: applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its *socket*
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
- A: no, *many* processes can be running on same host
- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - IP address: 128.119.245.12
 - port number: 80

App-layer protocol defines

- *types of messages exchanged,*
 - e.g., request, response
- *message syntax:*
 - what fields in messages & how fields are delineated
- *message semantics*
 - meaning of information in fields
- *rules* for when and how processes send & respond to messages

open protocols:

- defined in RFCs
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing



throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
text messaging	no loss	elastic	yes and no

Internet apps: application, transport protocols

application	application layer protocol	underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

Web and HTTP

- *web page* consists of *objects*
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects*
- each object is addressable by a *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

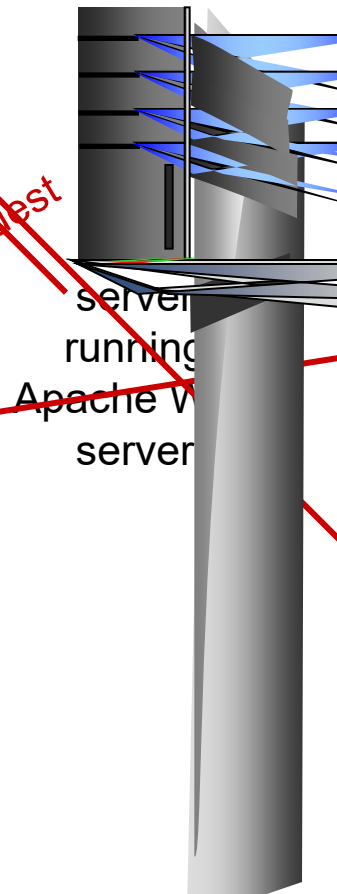
host
name

path
name

HTTP overview

HTTP: hypertext transfer protocol

- ❖ Web's application layer protocol
- ❖ client/server model
 - *client*: browser that requests, receives, (using HTTP protocol) and "displays" Web objects
 - *server*: Web server sends (using HTTP protocol) objects in response to requests



HTTP request

HTTP response

HTTP request

HTTP response

HTTP overview (cont..)

uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- server maintains no information about past client requests

aside
protocols that maintain “state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections

non-persistent HTTP

- at most one object sent over TCP connection
 - connection then closed
- downloading multiple objects required multiple connections

persistent HTTP

- multiple objects can be sent over single TCP connection between client, server

Non-persistent HTTP

suppose user enters URL: `www.someSchool.edu/someDepartment/home.index` (contains text, references to 10 jpeg images)

1a. HTTP client initiates TCP connection to HTTP server (process) at `www.someSchool.edu` on port 80

1b. HTTP server at host `www.someSchool.edu` waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object `someDepartment/home.index`

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

Non-persistent HTTP (cont.)



4. HTTP server closes TCP connection.

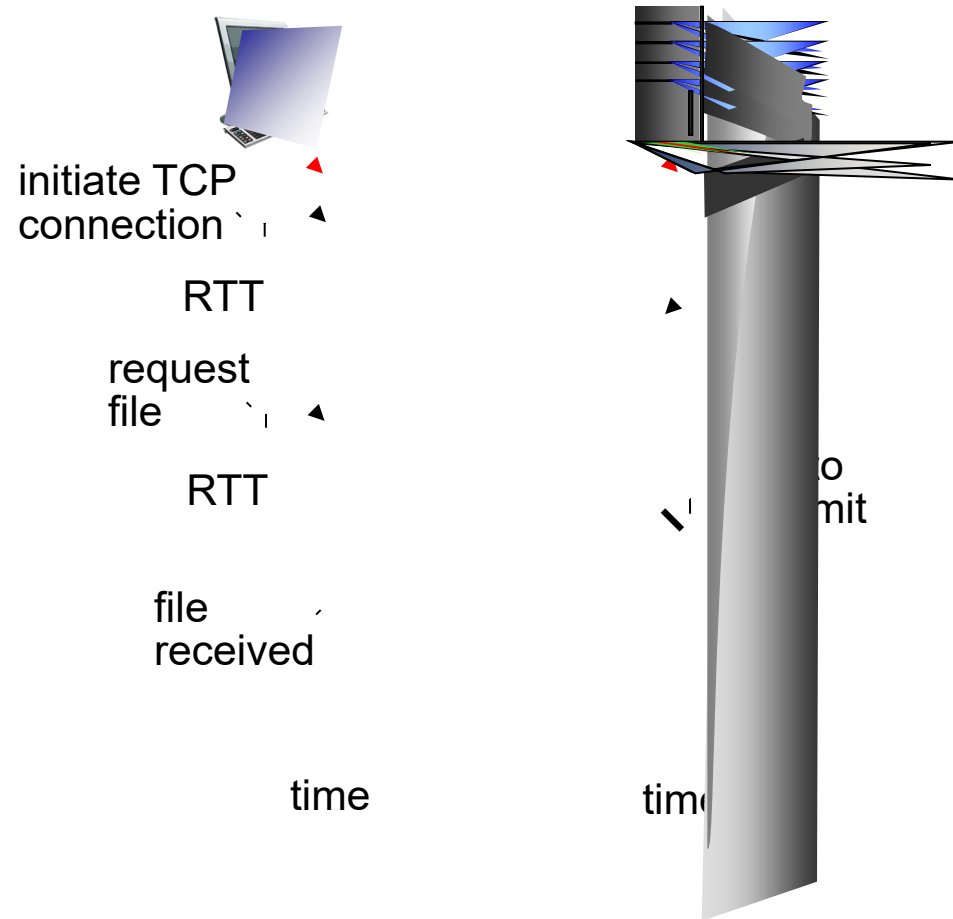
5. Steps 1-5 repeated for each of 10 jpeg objects

Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time
- non-persistent HTTP response time =
 $2\text{RTT} + \text{file transmission time}$



Persistent HTTP

non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open parallel TCP connections to fetch referenced objects

persistent HTTP:

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects

HTTP request message

- two types of HTTP messages: *request*, *response*
- HTTP request message:
 - ASCII (human-readable format)

request line
(GET, POST,
HEAD commands)

header
lines

carriage return,
line feed at start
of line indicates
end of header lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept:
    text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return
character
line feed
character

Method types

HTTP/1.0:

- GET
- POST
- HEAD
 - asks server to leave requested object out of response

HTTP/1.1:

- GET, POST, HEAD
- PUT
 - uploads file in entity body to path specified in URL field
- DELETE
 - deletes file specified in the URL field

HTTP response message

status line

(protocol ◀

status code

status phrase)

HTTP/1.1 200 OK\r\n

Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n

Server: Apache/2.0.52 (CentOS)\r\n

Last-Modified: Tue, 30 Oct 2007 17:00:02
GMT\r\n

header
lines

ETag: "17dc6-a5c-bf716880"\r\n

Accept-Ranges: bytes\r\n

Content-Length: 2652\r\n

Keep-Alive: timeout=10, max=100\r\n

Connection: Keep-Alive\r\n

Content-Type: text/html; charset=ISO-8859-
1\r\n

\r\n

data, e.g.,
requested
HTML file

data data data data data ...
◀

HTTP response status codes

- status code appears in 1st line in server-to-client response message.

- some sample codes:

200 OK

- request succeeded, requested object later in this msg

301 Moved Permanently

- requested object moved, new location specified later in this msg (Location:)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

User-server state: cookies

many Web sites use cookies

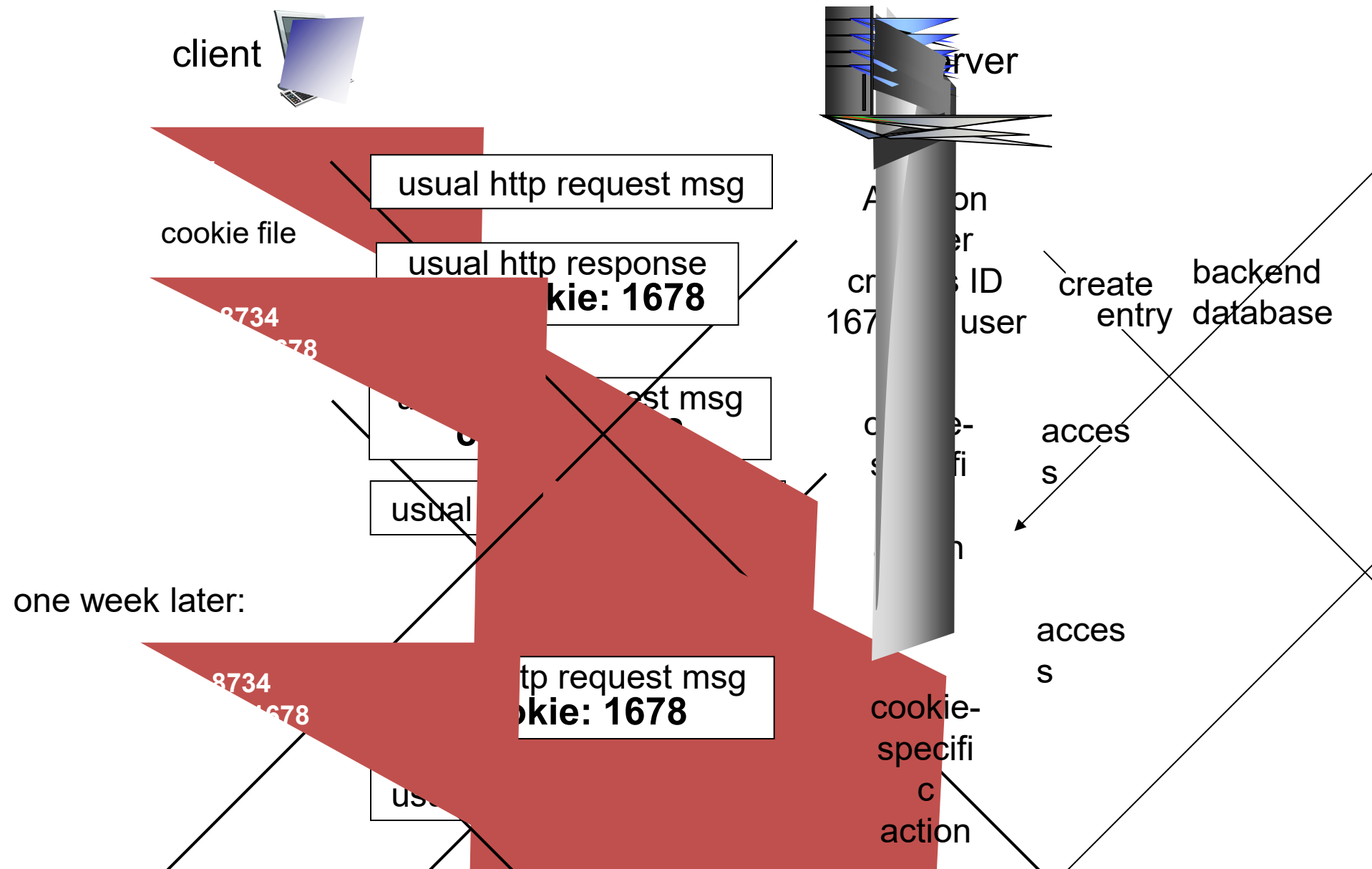
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

example:

- Susan always access Internet from PC
- visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID
 - entry in backend database for ID

Cookies: keeping “state” (cont.)



Cookies (cont..)

aside

what cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

cookies and privacy:

- cookies permit sites to learn a lot about you
- you may supply name and e-mail to sites

how to keep “state”:

- protocol endpoints: maintain state at sender/receiver over multiple transactions
- cookies: http messages carry state

More about Web caching

- cache acts as both client and server
 - server for original requesting client
 - client to origin server
- typically cache is installed by ISP (university, company, residential ISP)

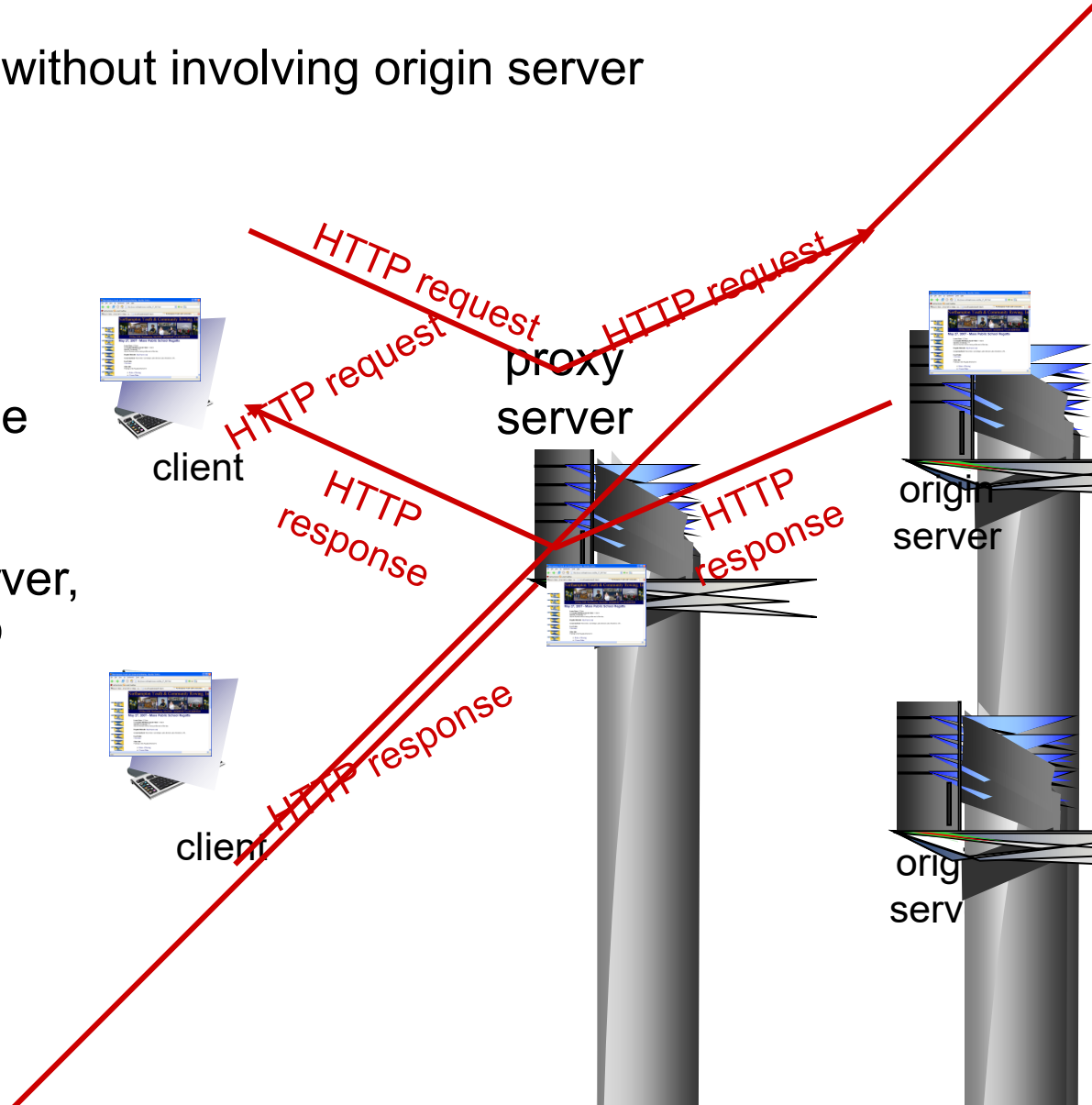
why Web caching?

- reduce response time for client request
- reduce traffic on an institution's access link
- Internet dense with caches: enables “poor” content providers to effectively deliver content (so too does P2P file sharing)

Web caches (proxy server)

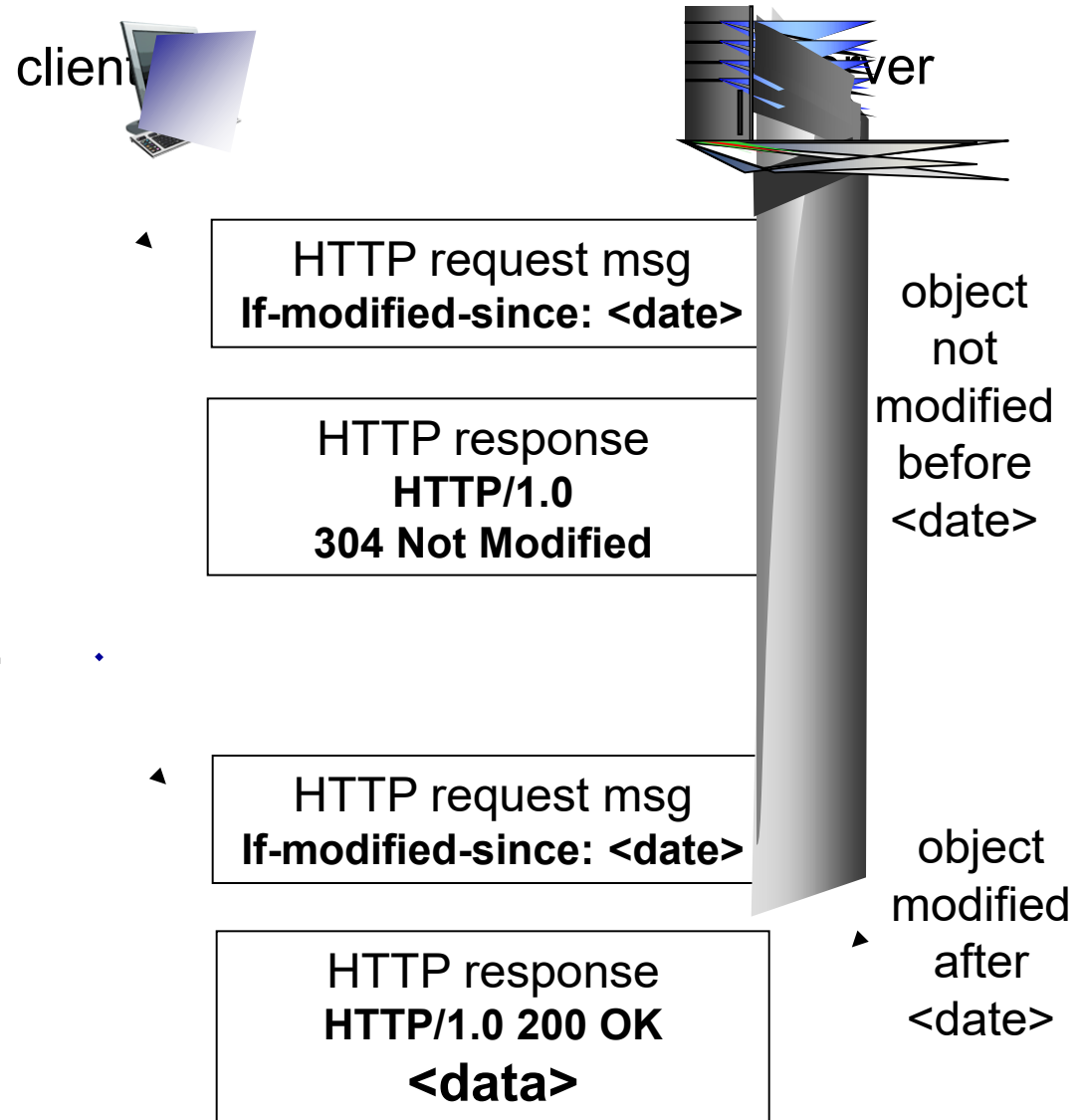
goal: satisfy client request without involving origin server

- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
 - object in cache: cache returns object
 - else cache requests object from origin server, then returns object to client



Conditional GET

- *Goal*: don't send object if cache has up-to-date cached version
 - no object transmission delay
 - lower link utilization
- *cache*: specify date of cached copy in HTTP request
If-modified-since: <date>
- *server*: response contains no object if cached copy is up-to-date:
HTTP/1.0 304 Not Modified



DNS: domain name system

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g.,
www.yahoo.com - used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System:

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol:* hosts, name servers communicate to *resolve* names (address/name translation)
 - note: core Internet function, implemented as application-layer protocol
 - complexity at network’s “edge”

DNS: services, structure

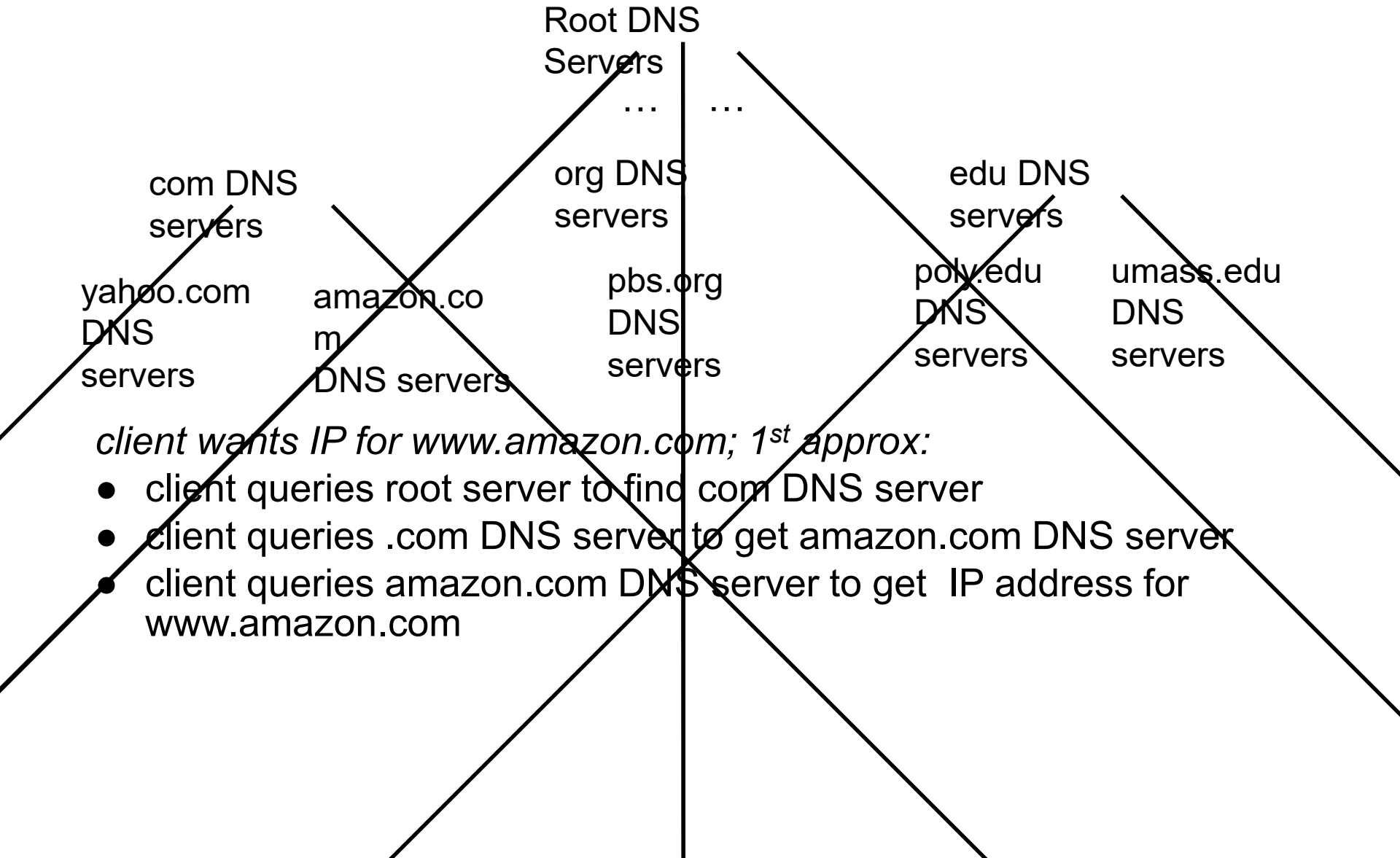
DNS services

- hostname to IP address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

why not centralize DNS?

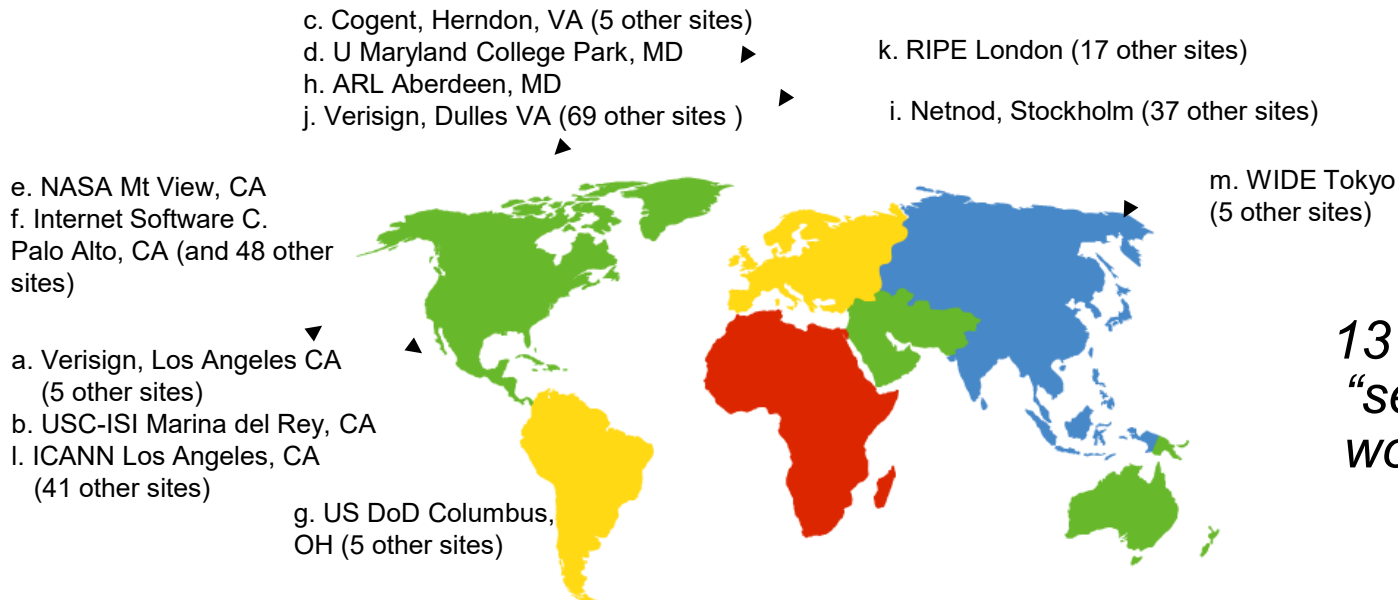
- single point of failure
- traffic volume
- distant centralized database
- maintenance

DNS: a distributed, hierarchical database



DNS: root name servers

- contacted by local name server that can not resolve name
- root name server:
 - contacts authoritative name server if name mapping not known
 - gets mapping
 - returns mapping to local name server



*13 root name
“servers”
worldwide*

TLD, authoritative servers

top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- Network Solutions maintains servers for .com TLD
- Educause for .edu TLD

authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name server

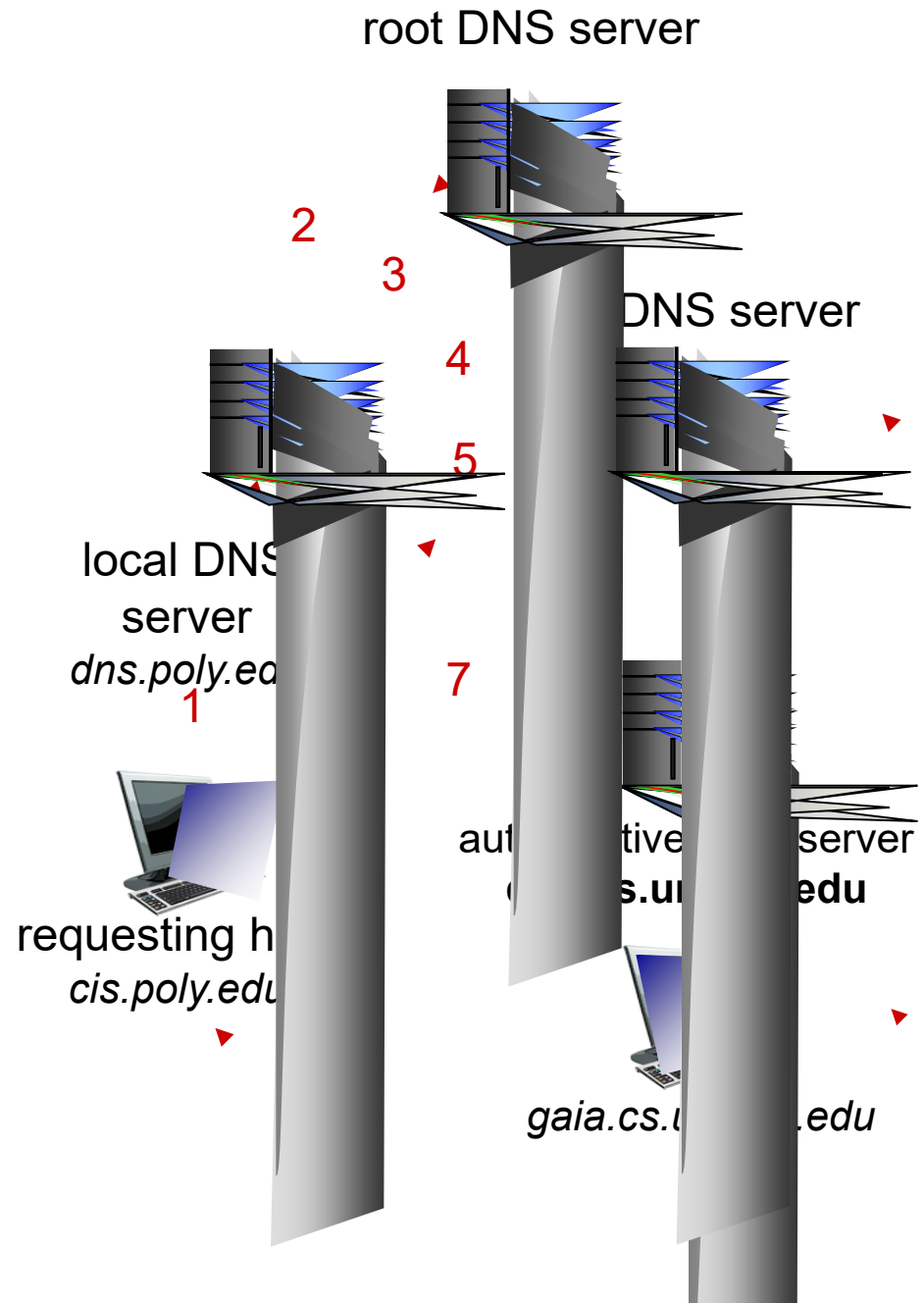
- does not strictly belong to hierarchy
- each ISP (residential ISP, company, university) has one
 - also called “default name server”
- when host makes DNS query, query is sent to its local DNS server
 - has local cache of recent name-to-address translation pairs (but may be out of date!)
 - acts as proxy, forwards query into hierarchy

DNS name resolution example

- host at cis.poly.edu wants IP address for gaia.cs.umass.edu

iterated query:

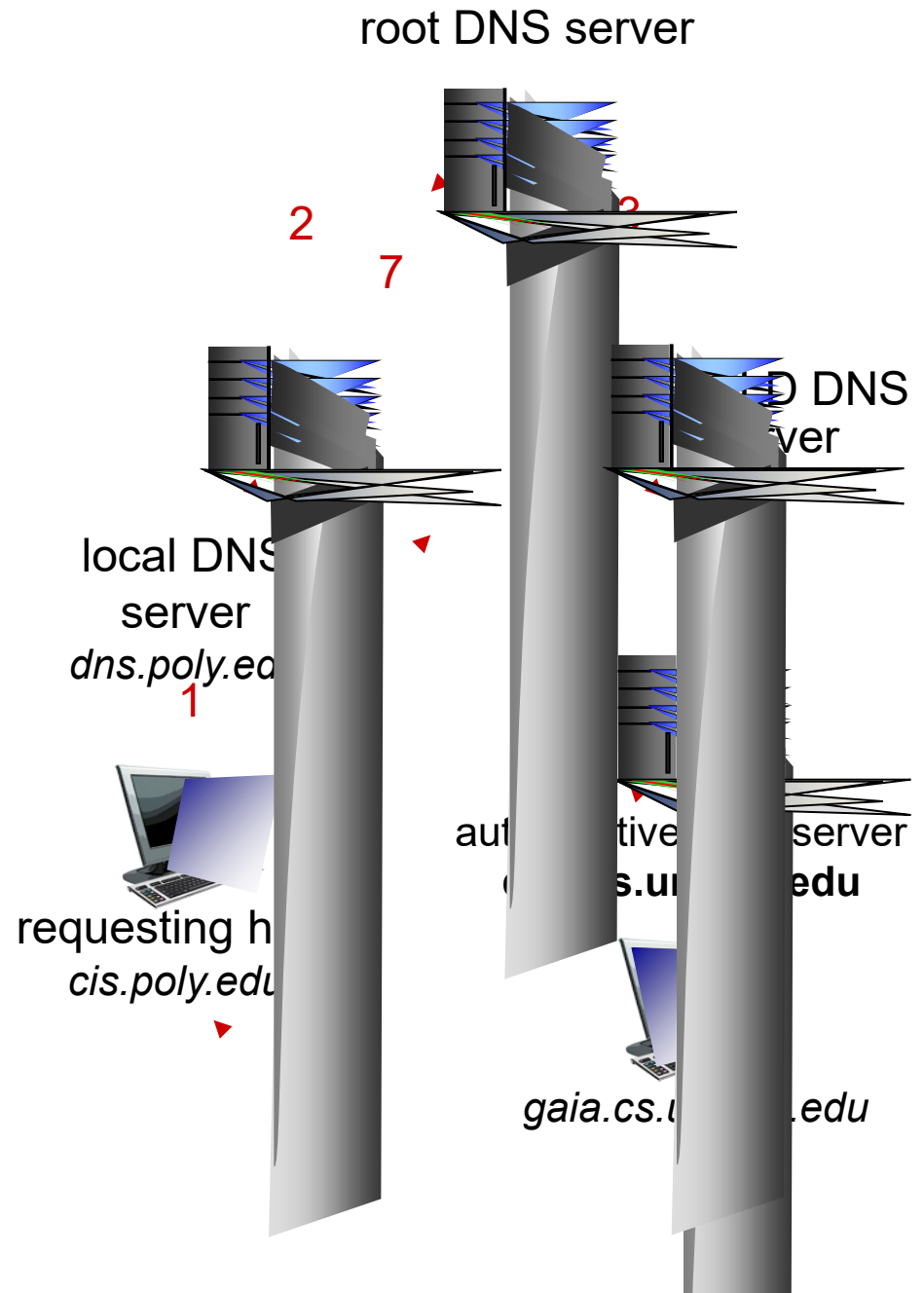
- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



DNS name resolution example

recursive query:

- ❖ puts burden of name resolution on contacted name server
- ❖ heavy load at upper levels of hierarchy?



DNS: caching, updating records

- once (any) name server learns mapping, it *caches* mapping
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
 - thus root name servers not often visited
- cached entries may be *out-of-date* (best effort name-to-address translation!)
 - if name host changes IP address, may not be known Internet-wide until all TTLs expire
- update/notify mechanisms proposed IETF standard
 - RFC 2136

DNS records

DNS: distributed db storing resource records (RR)

RR format: (name , value , type , ttl)
--

type=A

- **name** is hostname
- **value** is IP address

type=NS

- **name** is domain (e.g., foo.com)
- **value** is hostname of authoritative name server for this domain

type=CNAME

- **name** is alias name for some “canonical” (the real) name
- **www.ibm.com** is really **servereast.backup2.ibm.com**
- **value** is canonical name

type=MX

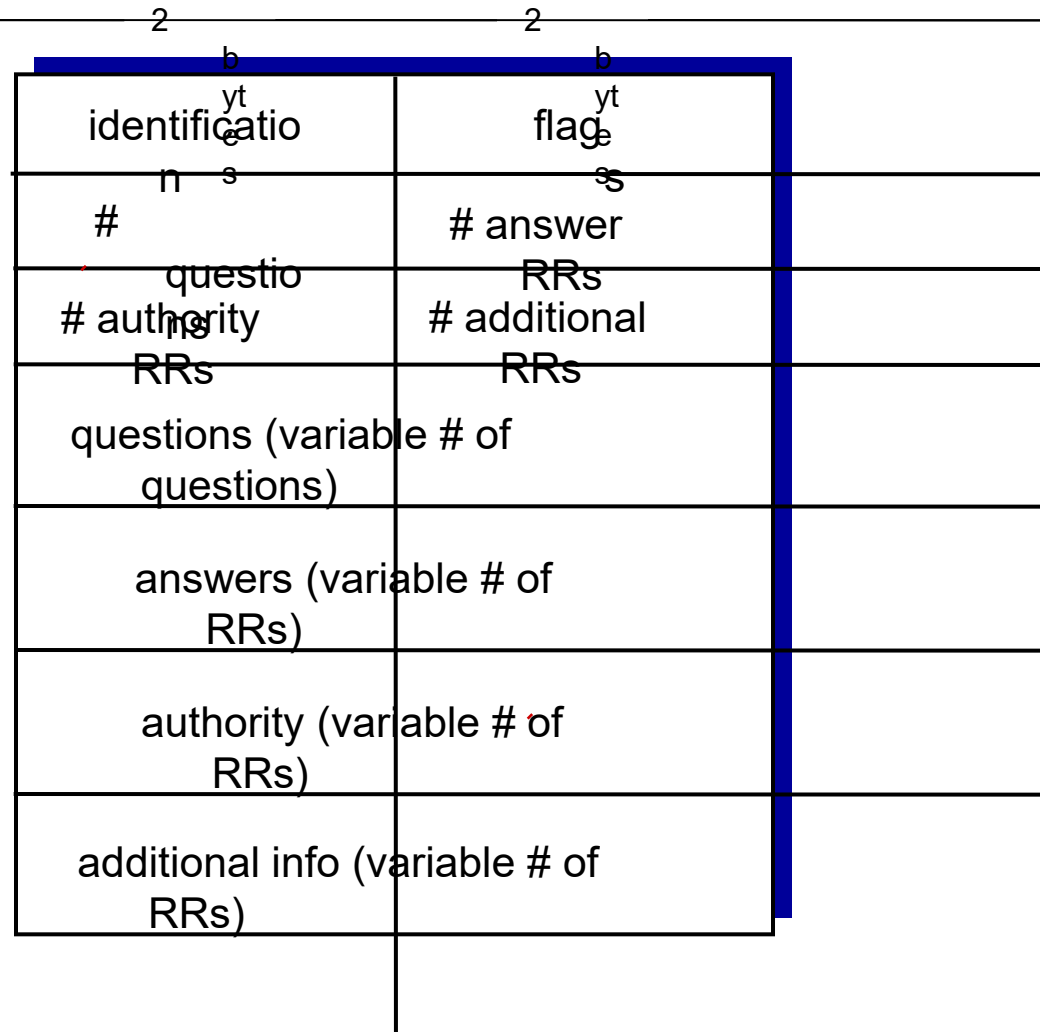
- **value** is name of mailserver associated with **name**

DNS protocol, messages

- *query* and *reply* messages, both with same *message format*

msg header

- identification: 16 bit # for query, reply to query uses same #
- flags:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



Attacking DNS

DDoS attacks

- Bombard root servers with traffic
 - Not successful to date
 - Traffic Filtering
 - Local DNS servers cache IPs of TLD servers, allowing root server bypass
- Bombard TLD servers
 - Potentially more dangerous

Redirect attacks

- Man-in-middle
 - Intercept queries
- DNS poisoning
 - Send bogus replies to DNS server, which caches

Exploit DNS for DDoS

- Send queries with spoofed source address: target IP
- Requires amplification

TCP: Overview

point-to-point:

one sender, one receiver

reliable, in-order *byte stream*:

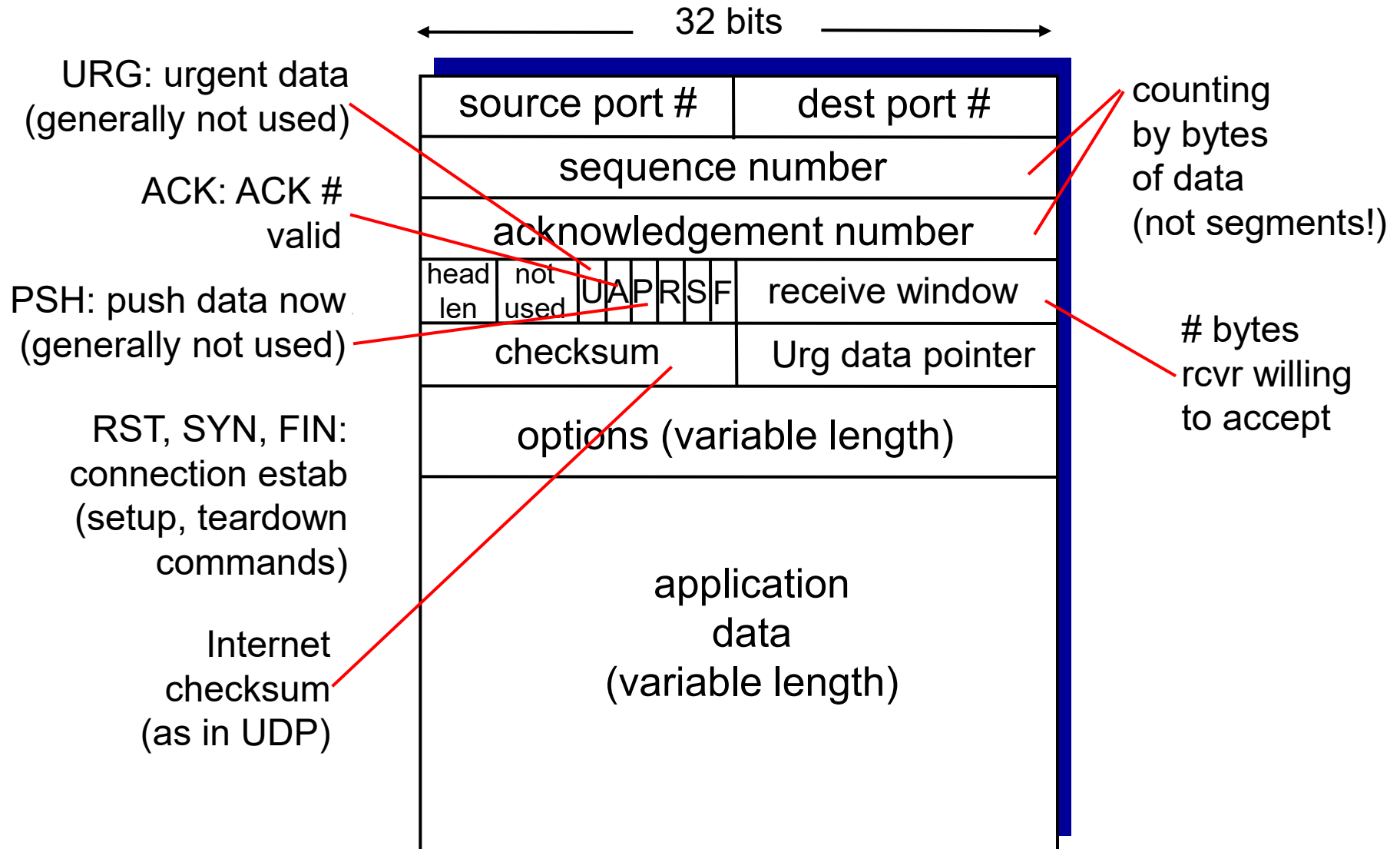
no “message boundaries”

pipelined:

TCP congestion and flow
control set window size

- full duplex data:
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- connection-oriented:
 - handshaking (exchange of control msgs) initializes sender, receiver state before data exchange
- flow controlled:
 - sender will not overwhelm receiver

TCP segment structure



TCP seq. numbers, ACKs

sequence numbers:

- byte stream “number” of first byte in segment’s data

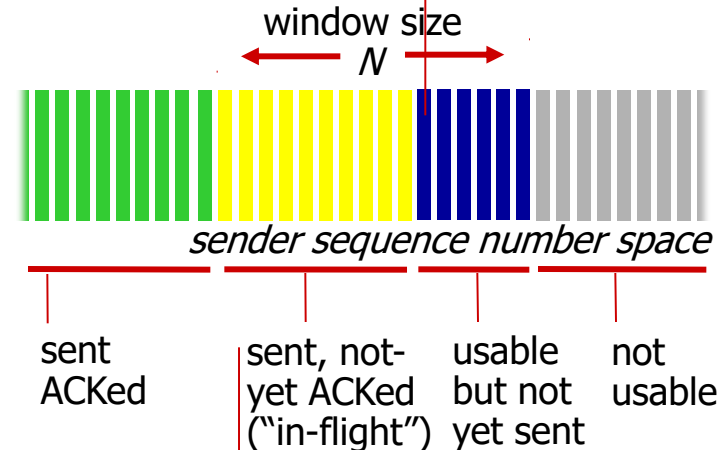
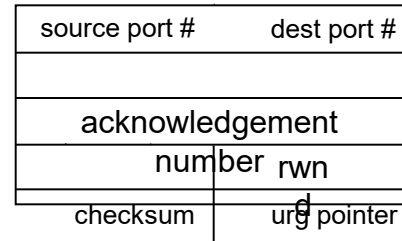
acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

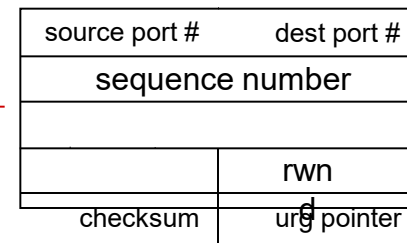
Q: how receiver handles out-of-order segments

- A:** TCP spec doesn’t say, - up to implementor

outgoing segment from sender



incoming segment to sender



TCP reliable data transfer

- TCP creates rdt service on top of IP's unreliable service
 - pipelined segments
 - cumulative acks
 - single retransmission timer
- retransmissions triggered by:
 - timeout events
 - duplicate acks

let's initially consider simplified TCP sender:
ignore duplicate acks
ignore flow control,
congestion control

TCP sender events:

data rcvd from app:

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
 - think of timer as for oldest unacked segment
 - expiration interval:
TimeoutInterval

timeout:

retransmit segment that caused timeout

restart timer

ack rcvd:

if ack acknowledges previously unacked segments

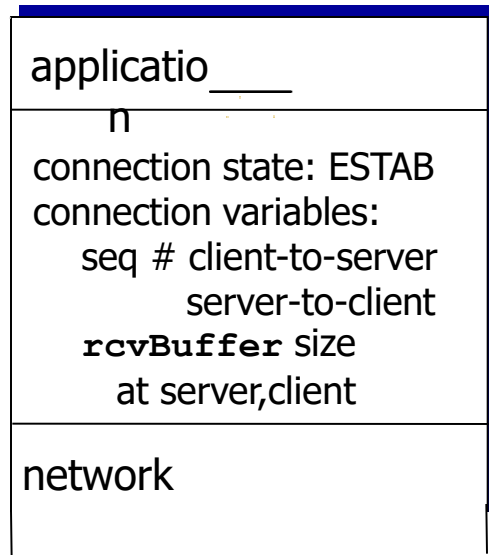
update what is known to be ACKed

start timer if there are still unacked segments

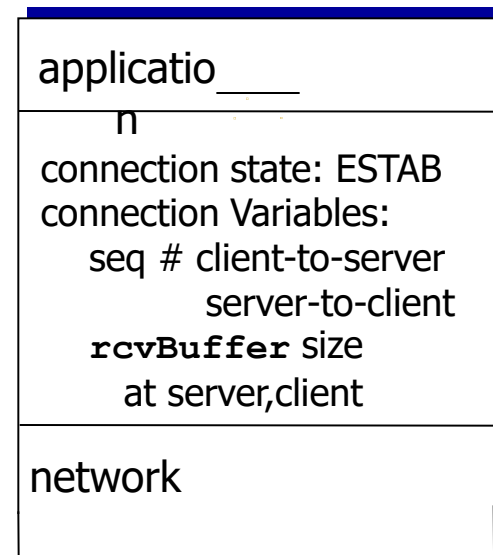
Connection Management

before exchanging data, sender/receiver “handshake”:

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters

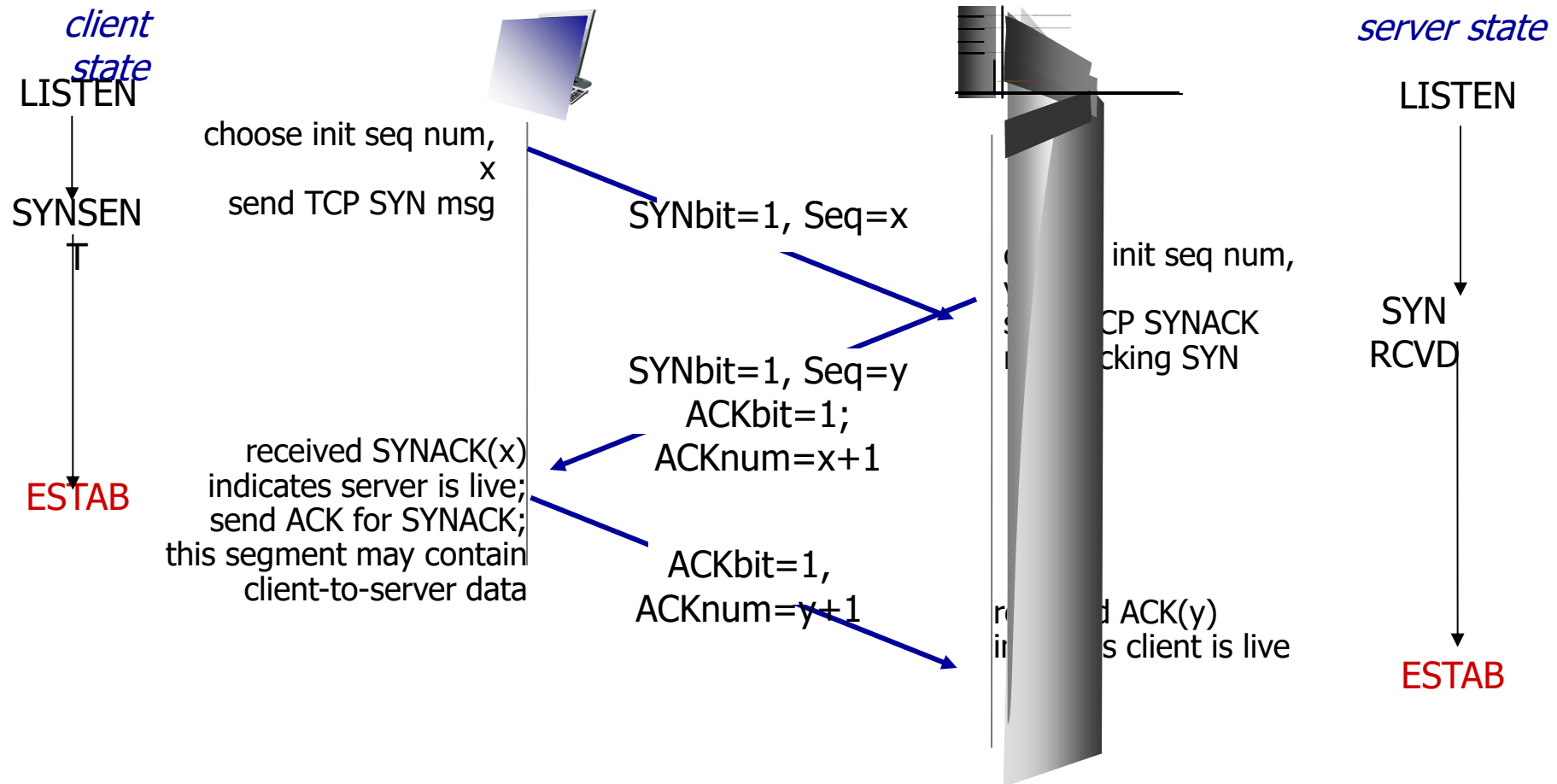


```
Socket clientSocket =  
    newSocket("hostname", "port  
    number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

TCP 3-way handshake



TCP: closing a connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

TCP: closing a connection

